Cadence Manual

June 20, 2026

Contents

1. *Cadence* Architecture and Execution Order

1.1. Architecture Overview

The *Cadence* simulator is built on a configuration-driven architecture in which simulation parameters are specified through configuration files loaded at runtime. The overall architecture, illustrated in Figure 1, is organised into configuration, physical modelling, instrumental response, and output generation stages.

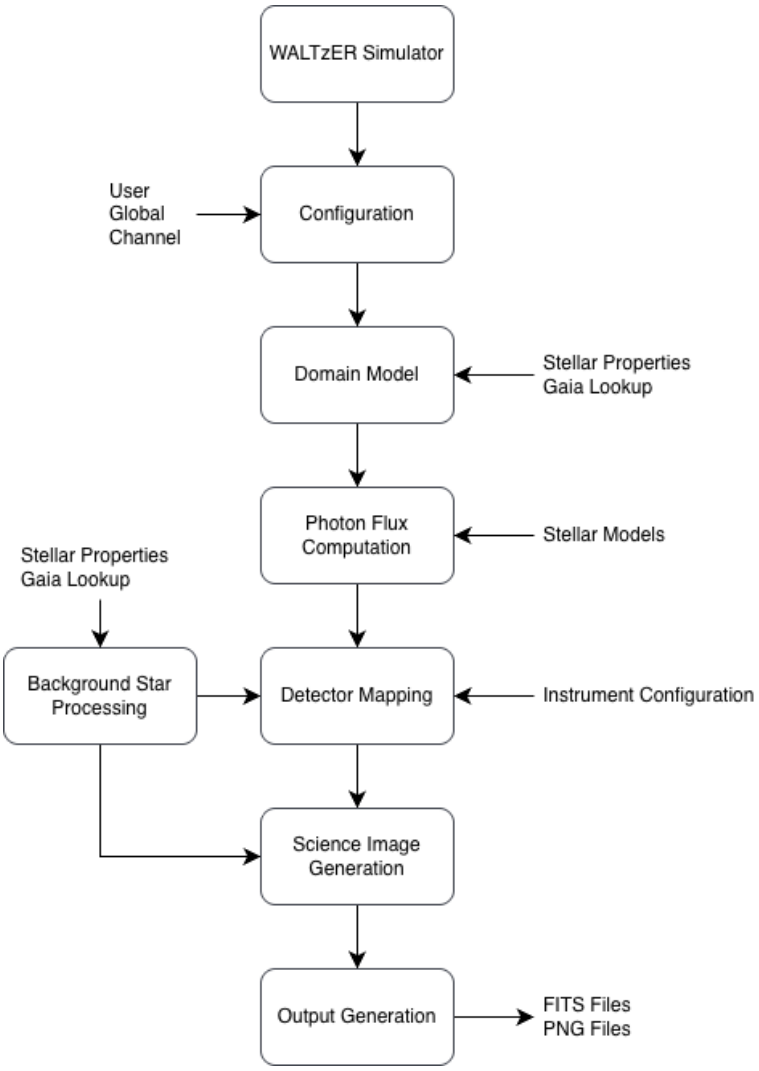

Figure 1: *Cadence* Architecture

Managed by the *Cadence* module, the execution follows a linear sequence: loading configurations, initialising the target object, computing photon flux, and generating science frames. While channels share the target star and orbital parameters, they are executed independently to account for unique exposure timings. This ensures that satellite rotation and detector-specific noise are calculated specifically for each channel's observation window.

1.2. Execution Pipeline

The top-level execution is defined in `cadence_simulator.py` and follows a sequence. The stages below correspond directly to the calls in `main()`.

1. **Runtime initialisation** — `initialize_cadence_runtime_context()` creates the timestamped output directory, sets up the log file, loads the global and user configuration, and constructs a `RunContext` containing the target name, output path, and run timestamp.
2. **Channel loading** — `load_channels_config()` reads the per-channel instrument configuration files and constructs the `NUV`, `VIS`, and `NIR` channel objects. Channels not enabled in `global.cfg` are set to `None`

and skipped in all subsequent steps.

3. **Target loading** — `load_stellar_and_planetary_properties()` resolves the target from the Excel catalogue, applies Gaia enrichment and parameter fallbacks (distance, radius, spectral type, $\log R'_{\mathrm{HK}}$), and returns the validated parameter dictionaries. `Star.from_params()` and `Planet.from_params()` construct the domain objects from these.

4. **Photon Flux Density on Earth** — `calculate_photon_flux_density_on_Earth()` derives the photon flux density for a given `Star` object from a photospheric model, applies optional line core emission and ISM absorption, converts to photon flux density in photons $\mathrm{s}^{-1}\,\mathrm{cm}^{-2}\,\mathrm{Å}^{-1}$, and returns it on a common wavelength grid covering all enabled channels. It is used for both the target star and background stars.

5. **Detector image preparation** — For each enabled channel, `prepare_detector_image_spectroscopy()` (NUV, VIS) and `prepare_detector_image_photometry()` (NIR) compute the raw target signal on the detector in electrons $\mathrm{s}^{-1}\,\mathrm{pixel}^{-1}$.

6. **Background star loading** — `lookup_background_stars()` loads or queries the background star catalogue. `populate_background_star_counts()` runs the flux pipeline and detector image preparation for each background star and caches the results in the `StarCatalog`.

7. **Science frame generation** — `build_science_images()` assembles and writes all science frames for a channel. Each frame combines bias, dark, flat-field, target signal, photon noise, diffuse background, cosmic rays, and background star contributions. Frames are written as FITS files; PNG outputs are optional.

Steps 4–7 are repeated independently per channel where noted, so that channel-specific exposure times, noise realisations, and detector geometries are applied correctly.

1.2.1. Runtime Initialisation

Runtime initialisation is handled by `initialize_cadence_runtime_context()` in `src/loaders/run_cadence_context.py`. It executes the following steps in order.

- `setup_output_directory()` creates a unique timestamped directory under `output/`. The timestamp is used as the directory name and as the basis for the log filename, ensuring that parallel or repeated runs never overwrite each other.
- `setup_logger()` creates the log file inside the output directory and configures Python's standard `logging` module. Any module in the codebase can then write to the log by importing `logging` directly, without needing access to the log file path or the `RunContext`. The complete simulation log is written there. Only user-relevant info (status reports of where we are) is additionally written to the console, as well as errors such as missing parameters or invalid target names. Detailed error information is in the log.
- `load_global_and_user_config()` loads both configuration files.
  - `load_global_config()` parses and validates `configs/global.cfg` (see Section 2.2.1) and caches the result as a singleton accessible via `get_global_config()` throughout the run (see Section 5.2).
  - `load_user_config()` loads the user parameter file (see Section 2.2.5) and caches it analogously, accessible via `get_user_config()` (see Section 5.3).

    `get_user_parameter_path()` determines which parameter file to use: if a path is passed as a command-line argument it is used directly, otherwise `input/parameters.txt` is loaded as the default. At most one argument is accepted; if too many arguments are passed or the file is missing, the simulator prints a usage message and exits with code 1.
- A frozen `RunContext` dataclass is constructed from the target name, output directory, and run timestamp. It captures the minimum shared context needed across pipeline stages, avoiding repeated passing of individual values, and is passed explicitly to all stages that write output or log target-specific information.

1.2.2. Channel Loading

Channel loading is handled in `load_channels_config()` located in `src/loaders/load_channel.py`. It accepts the `UserConfig` to read the per-channel exposure times, which are written directly into the channel objects during construction, as exposure time fits logically into a channel. Only channels enabled in `global.cfg` via `run_nuv`, `run_vis`, and `run_nir` are read and constructed; disabled channels are set to `None` and skipped in all subsequent pipeline stages.

- `load_channel_spectroscopy()` constructs a `SpectroscopyChannel` object for NUV and VIS from their respective configuration files `waltzer_nuv.cfg` and `waltzer_vis.cfg`.
  - `load_common_channel()` parses the shared detector parameters common to all channels: detector geometry, noise properties, gain, exposure time, and the effective area file.

    `load_effective_area_file()` loads the wavelength-dependent effective area and derives the pixel scale and spectral dispersion from the file header and wavelength grid.
  - `load_spread_profile_file_spectroscopy()` loads the wavelength-dependent cross-dispersion spread profile, or falls back to a Gaussian if none is configured.
  - `load_polarization_file()` loads the polarization delta file for spectropolarimetry mode if configured.
- `load_channel_photometry()` constructs a `PhotometryChannel` object for NIR from `waltzer_nir.cfg`, using the same shared parsing steps as above, plus `load_psf_image_file()` which loads and normalises the point spread function.
- `load_background_from_global_cfg()` loads the background spectrum or zodiacal emission data as configured in `global.cfg` and attaches it to the channel object, so it is available during science frame generation without reloading.

1.2.3. Target Loading

Target loading is handled by `load_stellar_and_planetary_properties()` in `src/loaders/load_stellar_and_planetary_properties.py`. It accepts the target name from the user configuration and returns validated parameter dictionaries for the star and planet, from which `Star.from_params()` and `Planet.from_params()` construct the domain objects.

- `load_matching_excel_row_from_excel()` locates the Excel catalogue in `input/` and finds the row matching the target name. The match is case-insensitive and strips the planetary suffix from the `pl_name` column before comparing.
- `map_to_planet_or_star_dictionary()` splits the matched row into separate stellar and planetary parameter dictionaries using the column mapping defined in `excel_mapping.cfg` (see Section 2.2.3).
- `get_missing_properties()` checks which required stellar parameters are still missing after the Excel import. If any can be provided by Gaia, `lookup_target_star_gaia()` queries Gaia DR3 for the target and returns the fetched parameters. `merge_gaia_into_star_params()` then merges only the missing values; Excel values always take precedence.
- `apply_radius_from_teff_mag_distance_if_missing()` estimates the stellar radius from the Gaia G magnitude, distance, and effective temperature if radius is not provided.
- `infer_mamajek()` assigns a spectral type from the Mamajek dwarf star table based on effective temperature.
- `apply_log_r()` assigns a fallback $\log R'_{\mathrm{HK}}$ value based on a temperature threshold defined in `global.cfg` if the activity index is not provided.
- `get_missing_properties()` is called once more after all fallbacks to check that no required parameters are still missing. If any are, the simulator stops and reports which parameters could not be resolved.
- `clean_and_cast_parameters()` normalises and type-casts all parameter values against the `Star` and `Planet` dataclass field definitions, so the domain objects can be constructed cleanly without any further parsing.

`Star.from_params()` and `Planet.from_params()` construct the domain objects from the validated parameter dictionaries. The planet object is currently not used in the simulation and is constructed for validation purposes only.

The target retrieval flow is illustrated in Figure 2.

1.2.4. Photon Flux Density on Earth

Photon flux computation is handled by `calculate_photon_flux_density_on_Earth()` in `src/instrument/prepare_detector_images.py`. It accepts the `Star` object, the `RunContext`, and the enabled channel objects. It returns the photon flux density in photons $\mathrm{s}^{-1}\,\mathrm{cm}^{-2}\,\text{Å}^{-1}$ on a wavelength grid covering all enabled channels. The internal steps of this pipeline are illustrated in Figure 3.

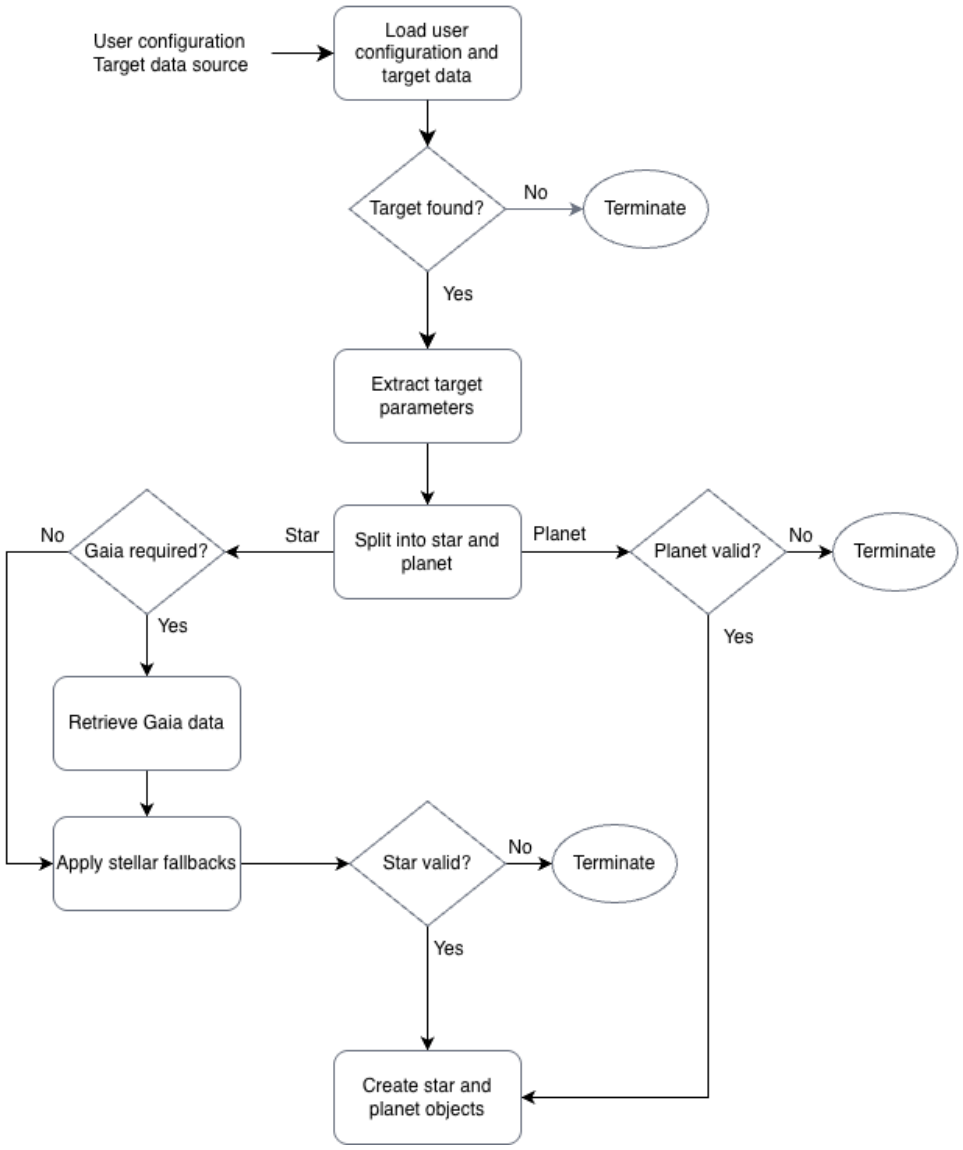

Figure 2: Target retrieval and validation flow.

- `get_required_wavelength_range()` derives the wavelength range spanning all enabled channels from their effective area grids, including a margin. The photospheric model is cut to this range so that all subsequent calculations operate only on the wavelengths relevant for the simulation.
- `run_photon_flux_density_pipeline()` executes the photon flux density pipeline for the given `Star` object on the trimmed wavelength grid.
  - `load_model_for_temperature()` selects the photospheric model closest to the target effective temperature and loads it from `data/models/`. Models are cached so repeated calls for the same temperature do not reload from disk (see Section 5.4).
  - `convert_stellar_model_to_flux()` converts the photospheric model from spectral flux density per unit frequency to per unit wavelength, integrates over the stellar sphere, and integrates over all directions.
  - `apply_line_core_emission()` adds Mg II h & k line core emission if enabled in `global.cfg`.
  - `compute_ebv_av()` computes the colour excess $E(B-V)$ from the target's galactic coordinates and distance using the Amores & Lépine extinction model.
  - `apply_ism_absorption()` applies interstellar medium absorption at the Mg II, Mg I, and Fe II resonance lines if enabled in `global.cfg`. Voigt profile shapes are cached to avoid recomputation across repeated calls (see Section 5.1).
  - `compute_flux_at_earth()` scales the stellar spectral flux density to the distance of the star.
  - `apply_unred()` corrects the spectral flux density for interstellar extinction caused by absorption and scattering of radiation by dust grains along the line of sight, using a parameterised extinction law.

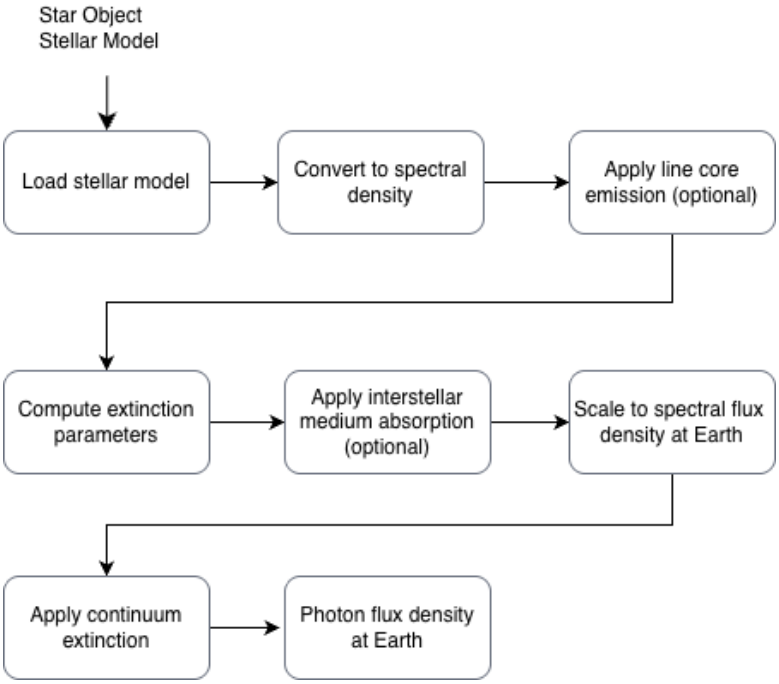

Figure 3: Photon flux density pipeline.

The extinction curve is cached per wavelength grid and extinction-law configuration (see Section 5.5).

– `convert_flux_to_photons()` converts the spectral flux density into a photon flux density.

1.2.5. Detector Image Preparation

`prepare_detector_image_spectroscopy()` is called for NUV and VIS. It accepts the photon flux, wavelengths, the channel object, the `RunContext`, and the `Star` object. It computes the 1D count rate for the channel, spreads it onto the 2D detector, and returns the resulting detector image.

- `compute_counts_per_s_px_one_channel()` handles the shared steps for all channels.
  - `compute_broadened_channel_flux()` cuts the photon flux to the channel wavelength range and convolves it with a Gaussian to match the instrument spectral resolution.
    - `cut_wavelength_window_with_margin()` trims the photon flux and wavelength arrays to the channel's effective area range plus a margin, so the subsequent convolution is not computed over the full wavelength grid.
    - `gaussbroad()` convolves the trimmed photon flux with a Gaussian of half-width equal to `channel.spectral_dispersion_A_per_pixel`, broadening the spectrum to the instrument's spectral resolution.
  - `counts_per_s_px_conv_per_channel()` interpolates the broadened flux onto the effective area wavelength grid, multiplies by the pixel scale and effective area, and returns the count rate in electrons s^{-1} pixel^{-1}.
- `spread_target_star_spectrum_to_2d()` determines the target's detector placement and spreads the 1D count rate onto the 2D detector.
  - `get_spectrum_placement()` returns the column, row, slope, and intercept describing where the target's spectrum is positioned on the detector.
  - `spread_1d_spectrum_to_2d()` is called when `observation_mode` is `spectroscopy`. Spreads the 1D count rate onto the 2D detector image at the given placement.
    - `_spread_1d_to_2d_profile()` is used when a wavelength-dependent cross-dispersion profile is configured for the channel. It distributes the counts across detector rows using the profile weights interpolated to the nearest spectral bin.
    - `_spread_1d_to_2d_gaussian()` is used otherwise, as a fallback. It distributes the counts across detector rows using a Gaussian with half-width `spread_half_height_pix`.

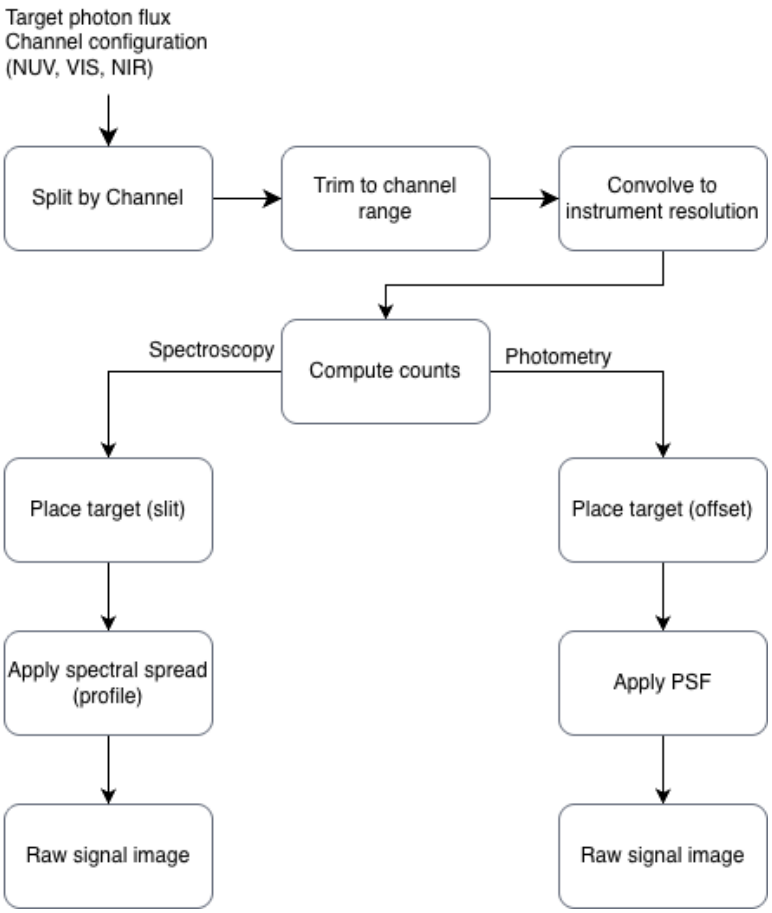

Figure 4: Detector mapping flow.

  - `spread_target_star_spectropolarimetry_to_2d()` is called when `observation_mode` is `spectropolarimetry`. Splits the counts into two polarization beams using the wavelength-dependent polarization delta, spreads each beam separately via `spread_1d_spectrum_to_2d()`, shifts the second beam by `beam_separation_pix` rows, and adds both beams into the detector image.

`prepare_detector_image_photometry()` accepts the same parameters for the NIR channel. It computes the 1D count rate and spreads it onto the 2D detector via the point spread function.

- `compute_counts_per_s_px_one_channel()` is the same function used for spectroscopy channels (see above); it handles the shared steps for all channels.
- `spread_1d_photometry_to_2d()` integrates the count rate over wavelength and places it on the detector using the PSF.
  - `get_photometry_placement()` converts the configured source offset in arcseconds to a detector pixel position, relative to the detector centre.
  - `paste_psf_stamp()` pastes the PSF stamp, scaled by the total flux, into the detector image at the computed position, clipping any part of the stamp that falls outside the detector boundary.

1.2.6. Surrounding fields / Background Stars

`lookup_background_stars()` accepts all channel objects, the `RunContext`, and the target `Star` object. It loads or queries the background star catalogue and constructs the `StarCatalog`.

- `_load_or_query_background_star_table()` returns a cached CSV table for the target if one exists in `data/BackgroundStars/`, named after the target without its planetary suffix. If no cached file is found, it queries Gaia DR3 within `gaia_conesearch_radius_arcsec` using a fixed magnitude cutoff of 20.0, independent of `magnitude_cutoff` in `global.cfg`, and saves the result as a CSV in the output directory. The fixed cutoff ensures the cached CSV always contains the full set of stars down to magnitude 20, so that a later run with a higher `magnitude_cutoff` does not silently end up with fewer background stars

than expected; the configured cutoff is instead applied afterwards as an in-memory filter.

  - `_load_background_csv_if_exists()` reads the cached CSV file from `data/BackgroundStars/` if present.
  - `gaia_lookup_for_background_stars()` performs the Gaia DR3 cone search if no cached CSV is found.
  - `_save_background_stars_csv()` writes the freshly queried table to the output directory for reuse in later runs.

- `_apply_background_star_magnitude_cutoff()` filters the loaded table in memory to stars at or below the configured `magnitude_cutoff` from `global.cfg`.
- `_annotate_background_star_offsets_arcsec()` computes each background star's angular offset and separation relative to the target, used later to determine whether and where it appears on the detector or in the slit.
- `_drop_stars_outside_max_radius()` removes stars that fall outside the maximum reach of any enabled channel's slit or field of view, so they are not carried through the remaining pipeline unnecessarily.
- `create_background_star_catalog()` constructs a `Star` object for each remaining background star, applying the same parameter fallbacks used for the target. Each star is added to the `StarCatalog` together with its offset and separation from the target, so the catalogue can later be iterated for flux and signal calculations.

`populate_background_star_counts()` accepts the `StarCatalog`, all channel objects, and the `RunContext`. For each background star in the catalogue, it computes the per-channel count rate and stores it in `counts_by_id_and_band` for later use when rendering science frames.

- `calculate_photon_flux_density_on_Earth()` runs the same photon flux pipeline used for the target (see Section 1.2.4) to obtain the background star's photon flux density.
- `spectroscopy_radius_arcsec()` computes the maximum reach of a spectroscopy channel's slit. If the star's separation from the target exceeds this radius, the star is skipped entirely for that channel, since it could never appear in the slit.
- `compute_counts_per_s_px_one_channel()` is the same function used for the target (see Section 1.2.5); it converts the background star's photon flux into a count rate for the channel.

For spectroscopy channels, the full 1D count-rate array returned by `compute_counts_per_s_px_one_channel()` is stored, since the star's position within the slit changes over the exposure as the field rotates. For the NIR photometry channel, the count rate returned by `compute_counts_per_s_px_one_channel()` is summed to a scalar total before storing, since photometry only requires the total flux for the PSF stamp rather than a per-pixel spectrum.

1.2.7. Science Frame Generation

`build_science_images()` is called once per enabled channel. It accepts the 2D target signal (NUV, VIS: spread along the dispersion direction; NIR: the PSF stamp pasted onto the detector), the channel object, the `RunContext`, the target `Star`, and the `StarCatalog` of background stars. It first builds a base FITS header shared by all frames for that channel, then generates calibration frames and science frames. The overall science frame assembly flow is illustrated in Figure 5.

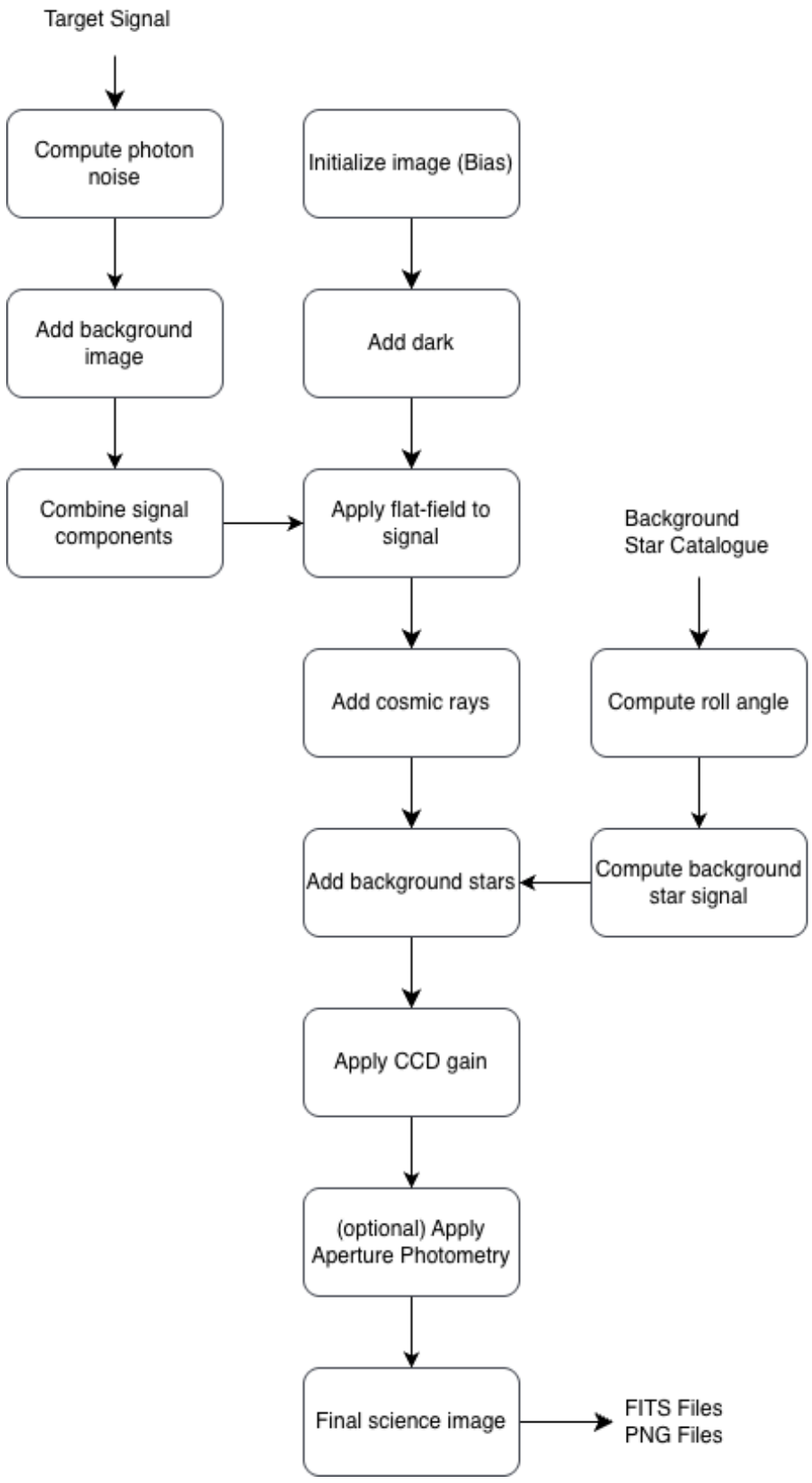

Figure 5: Science frame assembly flow.

- `initialize_fits_header()` creates a base FITS header containing the properties shared by all frames for the channel and target, such as target coordinates, magnitude, and exposure start time. Frame-specific header entries, for example bias or exposure time, are added later when each individual frame (bias, dark, flat, science) is generated.
- `_generate_channel_calibration_frames()` generates the configured number of bias, dark, and flat cali-

bration frames.

`_create_channel_images()` generates and writes all science frames for the channel. This is discussed in detail below.

Creating an Image per Channel `_create_channel_images()`

- The target signal is scaled by the exposure time, and the number of science frames (`n_science_frames`) is calculated from the orbit duration, number of revolutions, exposure time, and readout gap.
- `generate_background_image()` computes the diffuse background image once, since it is the same for all channels.
- For each frame, the roll angle range covered during that exposure is computed from the elapsed time since the start of the observation, the exposure time, and the readout gap, expressed as a fraction of the orbit duration and converted to degrees.
- `_create_per_exposure()` assembles the full per-pixel science image for the frame, combining bias, dark, flat-field, target signal, photon noise, diffuse background, cosmic rays, and background star contributions. As the central function of science frame generation, it is discussed in detail in the following paragraph.
- `compute_aperture_photometry()` computes aperture photometry statistics on the assembled frame, used only for the NIR channel.
- The frame header is finalised by appending the frame identity (`append_base_frame_header()`), image statistics (`append_image_stats_header()`), channel and exposure metadata (`append_channel_frame_header()`), and, where available, aperture photometry results (`append_photometry_header()`).
- `write_fits_frame()` writes the frame to a FITS file. `write_science_frame_png()` additionally writes a PNG if `write_science_frames_png` is enabled.

Per-Exposure Image Assembly `_create_per_exposure()` assembles the complete per-pixel science image for one exposure.

- `_build_science_image_without_bg_stars()` builds the image from all per-pixel components that do not depend on background stars.
  - `generate_bias_image()` initialises the image with the bias frame.
  - `generate_dark_image()` adds the dark current contribution.
  - `generate_photon_noise_from_spectra2d()` computes photon noise from the target signal.
  - `generate_flat_image()` generates the flat-field, which is applied multiplicatively to the combined target signal, photon noise, and diffuse background.
  - `generate_cosmic_rays()` adds randomised cosmic ray events.
- Depending on the channel type, `generate_background_star_spectroscopy_image()` (NUV, VIS) or `generate_background_star_photometry_image()` (NIR) renders the background star contributions for the given roll angle range and adds them to the image. See next paragraph for a description.
- The combined image is multiplied by `ccd_gain` to convert from electrons to the final detector units.

Background Star Rendering: Spectroscopy `generate_background_star_spectroscopy_image()` renders all background star contributions for one exposure into a 2D image, for the spectroscopy channels (NUV, VIS).

- `get_spectrum_placement()` determines the target's reference column and row on the detector once for the whole frame; background stars are positioned relative to this same reference point.
- `spectroscopy_radius_arcsec()` computes the slit's reach; stars beyond this separation are skipped before any per-star rendering.
- For each remaining star, `_render_star_if_in_slit()` determines whether and where the star falls within the slit during the exposure, and adds its contribution directly into the shared output image, so that all background stars end up combined into one image.
  - `compute_roll_angle_samples()` subdivides the exposure into small roll angle steps, fine enough that the star moves at most a fraction of a pixel per step.
  - `rotated_coordinates()` rotates the star's fixed angular offset from the target by each sampled roll angle.

  - `within_rotated_bounds_mask()` checks, for each roll angle sample, whether the rotated position falls within the slit half-width and half-length.
  - For each sample where the star is inside the slit, the corresponding detector row is computed and the time spent at that row is accumulated, since the star can stay at the same row across multiple consecutive samples.
  - `smear_1d_spectrum_dispersion()` applies the dispersion smear to the star's count rate, mirroring the same effect applied to the target's spectrum.
  - `spread_1d_spectrum_to_2d()` spreads the star's counts, scaled by the time spent at each row, onto the 2D image at that row. This is the same spreading function used for the target.

Background Star Rendering: Photometry `generate_background_star_photometry_image()` renders all background star contributions for one exposure into a single 2D image, for the NIR photometry channel.

- `get_photometry_placement()` determines the target's reference position on the detector once for the whole frame; background stars are positioned relative to this same reference point.
- For each star in the catalogue, `_render_star_if_on_detector()` determines whether and where the star falls on the detector during the exposure, and adds its contribution directly into the shared output image.
  - `compute_roll_angle_samples()` subdivides the exposure into small roll angle steps, fine enough that the star moves at most a fraction of a pixel per step.
  - `rotated_coordinates()` rotates the star's fixed angular offset from the target by each sampled roll angle, since the field of view rotates around the target over the orbit.
  - `within_rotated_bounds_mask()` checks, for each roll angle sample, whether the rotated position falls within the detector bounds.
  - For each sample where the star is on the detector, its detector position is computed, and the cached total flux is divided equally across all such samples, since the star is treated as visible for the full exposure once it appears at least once.
  - `paste_psf_stamp()` pastes the PSF stamp, scaled by the per-step flux, into the shared image at each computed position. This is the same pasting function used for the target.

2. Setup and Installation

Clone the repository and install all required packages (easiest, needed for tests on github):

```
pip install -r tests/config/requirements.txt
```

Some features require network access at runtime, specifically Gaia archive lookups and SIMBAD name resolution via `astroquery`.

Required packages: `numpy`, `scipy`, `matplotlib`, `astropy`, `astroquery`, `openpyxl`, `pandas`, `pytest`.

2.1. Repository Structure

- `configs/` — global and per-channel instrument configuration files:
  - `global.cfg` — global simulation settings
  - `waltzer_nuv.cfg`, `waltzer_vis.cfg`, `waltzer_nir.cfg` — per-channel instrument configuration
  - `excel_mapping.cfg` — mapping between Excel catalogue columns and internal parameter names
- `data/` — all static data files required by the simulator:
  - `waltzer_nuv_eff_area.txt`, `waltzer_vis_eff_area.txt`, `waltzer_nir_eff_area.txt` — effective area curves per channel
  - `nir_psf.txt` — NIR point spread function
  - `nuv_cross_dispersion_spread.txt` — NUV cross-dispersion spread profile
  - `nuv_spectropolarimetry.txt`, `vis_spectropolarimetry.txt` — polarization delta files for spectropolarimetry mode
  - `background.txt` — default background spectrum
  - `zod_dist.txt`, `zod_spectrum.txt` — zodiacal light distribution and spectrum
  - `stellar_param_mamjeck.txt` — Mamajek star parameter table for spectral type assignment
  - `models/` — stellar atmosphere model grids
  - `BackgroundStars/` — cached Gaia background star catalogues per target
- `input/` — target parameter files and Excel target catalogue:
  - `Targets_V10p1.xlsx` — Excel catalogue of targets and stellar parameters
  - `parameters.txt` — default parameter file loaded when no argument is given
  - `parameters/` — additional parameter files for specific targets
- `output/` — simulator output, one subdirectory per run
- `src/` — all simulator source code
- `tests/` — unit tests and snapshot tests

2.2. Configuration

Cadence uses a combination of configuration files, runtime parameters and source-code constants. These are stored in different locations within the project:

- `configs/`: simulator and instrument configuration files (`global.cfg`, `excel_mapping.cfg`, `waltzer_*.cfg`)
- `input/`: run-specific user input (`parameters.txt`)
- `src/utils/Constants.py`: implementation constants used throughout the simulator

The simulator distinguishes between constants, configuration files and runtime parameters. The distinction is primarily based on how frequently values are expected to change and how much validation is applied when they are modified.

2.2.1. `global.cfg`

Global simulation settings: pipeline control (which channels to run), timing and observation parameters, Gaia configuration, ISM and line core emission parameters, cosmic ray settings, background configuration, and output control.

These values are configurable and validated during loading. They are expected to change occasionally between simulation runs, but are generally stable during normal operation.

For details on configuration loading, caching, and runtime access through `get_global_config()`, see Section 5.2.

2.2.2. `waltzer_nuv.cfg`, `waltzer_vis.cfg`, `waltzer_nir.cfg`

Per-channel instrument configuration: detector geometry, noise properties, effective area file, slit or PSF properties, and spread profile.

The files retain the `waltzer` naming convention because they describe a specific instrument configuration rather than the Cadence simulator itself. Cadence is intended to be instrument-independent, while these configuration files contain mission- or instrument-specific parameters.

2.2.3. `excel_mapping.cfg`

The Excel mapping file decouples the Excel catalogue column names from the internal parameter names used by the simulator. Without it, column name changes in the catalogue would require modifying the source code. Instead, only the mapping file needs to be updated.

The file also separates planetary and stellar properties, so the simulator can assign parameters to the correct class (`Star` or `Planet`) without hardcoding this distinction. Finally, it defines which parameters are required. For the primary target, if any required parameter is missing after all enrichment steps — including Gaia lookup and fallback derivations — the simulator stops and reports which parameters are missing. For background stars, missing parameters are handled differently: since Gaia does not always provide a complete set of properties and not all of them can be derived, background stars with missing required parameters are silently skipped rather than stopping the simulation.

2.2.4. Constants (src/utils/Constants.py)

Constants are defined directly in the source code and are generally not intended to be modified during normal simulator operation. They typically represent implementation decisions, fixed assumptions or values that affect the behaviour of multiple pipeline stages.

Constants are not exposed through configuration files and are generally not heavily guarded against invalid modifications. Changing them assumes an understanding of the affected code and the potential consequences for the simulation pipeline.

2.2.5. `parameters.txt` / User Config

The parameter file contains the runtime inputs selected by the user for a specific simulation run. At the moment this includes the target name, the total observation length, and the exposure times for the individual channels.

The `target_name` is the critical field. It is used to select the target system from the Excel catalogue. Its properties are then read from the catalogue, and missing values can be retrieved from Gaia where implemented. Because the target name controls which system is simulated, it is strongly validated during loading and preprocessing.

The exposure times are read from `parameters.txt` and written into the channel objects during channel initialisation. They are not treated as global configuration values afterwards.

`parameters.txt` is therefore not a general simulator configuration file. It defines the concrete run setup: which target is simulated and with which exposure times.

2.3. Data Files

2.3.1. Effective Area Files

The effective area files for the three channels (`waltzer_nuv_eff_area.txt`, `waltzer_vis_eff_area.txt`, `waltzer_nir_eff_area.txt`) can be replaced with updated instrument models as long as the file format is preserved.

The file must contain the following:

- A header line `# Pixel scale: <value>` specifying the spatial pixel scale in arcseconds per pixel.
- A whitespace-delimited numeric table where the first column is wavelength in Angstrom and the last column is effective area in cm^2. Intermediate columns are ignored.

The wavelength grid in the effective area file defines the detector pixel grid. The broadened photon flux is interpolated onto this grid, multiplied by the effective area, and multiplied by the pixel scale to produce counts per second per pixel. For spectroscopy channels (NUV and VIS), the number of wavelength rows must match `x_pixels` in the channel configuration exactly, as each row corresponds to one detector pixel in the dispersion direction. For the NIR photometry channel no such constraint applies.

The spectral dispersion in Å per pixel is derived from the wavelength step between the first two rows of the table and stored as `spectral_dispersion_A_per_pixel` on the channel. This value is used for the Gaussian broadening of the stellar spectrum before convolution with the instrument response.

The wavelength range of each channel is entirely determined by the effective area file. There is no separate configuration for it.

2.3.2. Cross-Dispersion Spread Profile

The spread profile to use is configured per channel via `spread_profile_file`. If a file is given, the wavelength-dependent profile is loaded and used. If `spread_profile_file` is empty and `spread_half_height_pix` is set, a Gaussian fallback is used instead. If neither is configured, the simulator stops with an error.

The spread file format is a header row starting with `pixels` followed by wavelength values in Angstrom, then one row per pixel offset containing the fractional weight at each wavelength. The number of wavelength bins in the spread file does not need to match the detector pixel count — the simulator finds the nearest bin for each detector pixel and multiplies the counts by the corresponding weights to build the 2D image.

2.3.3. NIR PSF File

The PSF file is configured via `psf_file` in `waltzer_nir.cfg`. It contains a 2D numeric grid, normalised to sum to 1 after loading. The total counts are summed into a scalar, the PSF is multiplied by it, and the result is pasted onto the detector image at the source position. Portions of the PSF stamp falling outside the detector boundary are clipped.

3. Running the Simulator

Run from the repository root or `src` in all cases.

Default parameter file The simplest invocation loads `input/parameters.txt` as the default parameter file:

```
python src/cadence_simulator.py
```

Specific parameter file A specific parameter file can be passed as an argument:

```
python src/cadence_simulator.py input/parameters/parameter_001.txt
```

Batch run To run all `.txt` parameter files in a folder sequentially, use the batch runner script:

```
input/starrunner.sh input/roland
```

The script iterates over all `.txt` files in the given directory and runs the simulator for each one in sequence. Without an argument it iterates over all subdirectories of the current directory. The exit code of each run is reported.

3.1. Example Runs

3.1.1. Default Parameter File

The simplest invocation loads `input/parameters.txt` as the default parameter file. Run with:

```
python src/cadence_simulator.py
```

`input/parameters.txt` contains:

```
# Name of the target star. Its properties are read from the Excel file in the project root.
# If a property is missing there, it will be retrieved from GAIA.
target_name = KELT-9

# Total duration of the simulated observation, in hours.
# The transit is centered in this window, with half of the time before and half after the
# transit.
# Floating-point values are allowed (e.g. 24 or 24.5).
# not implemented yet
total_observation_length_h = 11.11

# Exposure time of the Channels in seconds
exposure_NUV_s = 300.0
exposure_VIS_s = 60.0
exposure_NIR_s = 60.0
```

This produces a new timestamped output directory, for example: `output/20260620_142936_214098/` containing:

```
cadence_simulator_20260620_142936_214098.log
WALTzER_KELT-9_NUV_science_300s_00000.fits
WALTzER_KELT-9_NUV_science_300s_00000.png
WALTzER_KELT-9_VIS_science_60s_00000.fits
WALTzER_KELT-9_VIS_science_60s_00000.png
WALTzER_KELT-9_NIR_science_60s_00000.fits
WALTzER_KELT-9_NIR_science_60s_00000.png
...
```

with one FITS/PNG pair per science frame and channel. The full directory structure, including calibration frames and diagnostic plots when enabled, is described in Section 3.2.

3.1.2. Specific Parameter File

A specific parameter file can be passed as an argument:

```
python src/cadence_simulator.py input/parameters/parameter_001.txt
```

`input/parameters/parameter_001.txt` contains the same structure as `input/parameters.txt` described above, but with different target and exposure values, for example:

```
target_name = HD 2685
total_observation_length_h = 11.11
exposure_NUV_s = 300.0
exposure_VIS_s = 60.0
exposure_NIR_s = 60.0
```

This produces a new timestamped output directory, for example: `output/20260620_143405_439029/` containing:

```
cadence_simulator_20260620_143405_439029.log
WALTzER_HD_2685_NUV_science_300s_00000.fits
WALTzER_HD_2685_NUV_science_300s_00000.png
WALTzER_HD_2685_VIS_science_60s_00000.fits
WALTzER_HD_2685_VIS_science_60s_00000.png
WALTzER_HD_2685_NIR_science_60s_00000.fits
WALTzER_HD_2685_NIR_science_60s_00000.png
...
```

with the same structure as described for the default parameter file above.

3.1.3. Batch Run

To run all `.txt` parameter files in a folder sequentially, use the batch runner script.

```
bash input/starrunner.sh [directory]
```

`starrunner.sh` is a Bash script and has not been adapted for Windows; it must be run from a Unix-like shell (Linux, macOS, WSL, or Git Bash). It supports two modes.

Batch Run Modes Given the following directory structure:

```
input/exposures/
    3460_11_D/
        001.txt  002.txt  003.txt  004.txt  005.txt  006.txt
    5800_11_D/
        001.txt  002.txt  003.txt  004.txt  005.txt  006.txt
    5800_11_G/
        001.txt  002.txt  003.txt  004.txt  005.txt  006.txt
```

Run a single leaf directory: `bash input/starrunner.sh input/exposures/3460_11_D`.

This runs all 6 parameter files in `3460_11_D/` only.

Run all subdirectories of a given directory: `bash input/starrunner.sh input/exposures`

This runs all 6 parameter files in `3460_11_D/`, then all 6 in `5800_11_D/`, then all 6 in `5800_11_G/`.

Each parameter file produces its own independently timestamped output directory under `output/`, regardless of how it was invoked. Since the output directory name is generated from the run timestamp and not from the parameter filename, running many files with generic names such as `001.txt` across many folders is safe – it does not cause output collisions. The script reports the exit code of each run and continues to the next file even if one fails.

3.2. Output Directory Structure

Each simulation run creates a unique timestamped directory under `output/`. The directory contains the following.

- `cadence_simulator_<timestamp>.log` — the complete log file for the run.
- `<target_name>.csv` — the background star table fetched from Gaia for this target, if no cached CSV was already available in `data/BackgroundStars/` (see Section 1.2.6).
- `WALTzER_<target_name>_<channel>_science_<exposure>s_<index>.fits` — one science frame per exposure and channel. These are the actual simulated science output, written by `write_fits_frame()` (see Section 1.2.7).
- `WALTzER_<target_name>_<channel>_science_<exposure>s_<index>.png` — a PNG rendering of the corresponding science frame, written by `write_science_frame_png()` if `write_science_frames_png` is enabled (see Section 1.2.7).
- `<target_name>_*.png` — diagnostic and intermediate plots from the flux pipeline (e.g. model input, line core emission, ISM absorption, photon flux) and the detector convolution step, written when `produce_flux_convolution_plots` is enabled. These are not science output and are intended for verifying intermediate pipeline steps.

Calibration frames (bias, dark, flat), if generated, follow the same naming pattern as science frames but with `bias`, `dark`, or `flat` in place of `science`. All FITS files, whether calibration or science, carry the `WALTzER_` prefix. For PNGs, only science frames carry the prefix; calibration frame PNGs do not.

- `WALTzER_HD_189733_NUV_bias_300s_00000.fits` — a bias calibration frame for the NUV channel.
- `WALTzER_HD_189733_NUV_bias_300s_00000.png` — the same bias calibration frame, written in PNG format.
- `WALTzER_HD_189733_NUV_science_300s_00000.fits` — the first science frame for the NUV channel.
- `WALTzER_HD_189733_NUV_science_300s_00000.png` — the PNG rendering of that same science frame.
- `HD_189733_Detector_counts_s_px_convolved_NUV.png` — a diagnostic plot of the convolved detector count rate for the NUV channel, not prefixed.
- `HD_189733.csv` — the cached background star table for the target.

3.3. Excel Target Catalogue

The simulator searches the `input/` directory for an Excel file at startup and loads whichever one it finds. Temporary lock files created by Excel (e.g. `~$Targets.xlsx`) are automatically ignored. Only one real Excel file may be present — placing a second one there will cause the simulator to stop with an error. The first active worksheet is used, so if the catalogue ever contains multiple worksheets, the target data must be on the first one.

The simulator looks up the target by matching the `target_name` from the parameter file against the `pl_name` column. The `pl_name` column contains planet names including the planetary suffix (e.g. `KELT-9 b`), while `target_name` in the parameter file is specified without the suffix (e.g. `KELT-9`). The simulator strips the last character from each `pl_name` entry before comparing. Target names are trimmed of whitespace, lowercased, and compared case-insensitively, so minor formatting differences including leading or trailing spaces and differing capitalisation do not cause lookup failures.

If target initialisation fails despite the name appearing correct, check the `pl_name` entry in the Excel file carefully. Excel sometimes introduces hidden characters when cell contents are copied from external sources such as websites or other documents. These are not visible in the cell but will cause the lookup to fail. To find them, paste the cell contents into a plain text editor — hidden characters will become visible there. Retyping the name manually in Excel can also resolves this.

3.4. Gaia Helper Scripts

The `src/gaiahelper/` directory contains two helper scripts that are not part of the Cadence simulation pipeline but were used to pre-fetch and organise background star data from the Gaia archive.

`gaiahelper_create_background_stars.py` Fetches background stars from the Gaia DR3 archive for a list of targets and saves one CSV file per target into a specified output directory. The Gaia query used is identical to the one used internally by Cadence, so the resulting CSV files are directly compatible with `data/BackgroundStars/`.

Pre-fetching background stars with this script and placing the CSVs there is strongly recommended — Gaia queries during a simulation run are slow and can take a very long time for crowded fields.

`generate_master_excel.py` Combines all CSV files in a directory into a single Excel file for inspection. Useful for getting an overview of all pre-fetched background stars across targets.

3.5. Gaia Lookups

Gaia is queried in two situations during a simulation run.

Target star If required stellar parameters are missing from the Excel catalogue after loading, the simulator queries Gaia DR3 for the target and fetches all available parameters. Only those missing from the Excel catalogue are then merged into the parameter set — Excel values always take precedence and are never overwritten by Gaia.

Background stars If no CSV file for the target is found in `data/BackgroundStars/`, the simulator queries Gaia to fetch background stars at runtime. The resulting CSV is saved to the output directory and can be moved to `data/BackgroundStars/` for subsequent runs. It is strongly recommended to pre-fetch background stars using `gaiahelper_create_background_stars.py` before running the simulator. Gaia queries are slow, unreliable, and can take a very long time for crowded fields.

The Gaia fetch always uses a fixed magnitude limit of 20.0, independent of `magnitude_cutoff` in `global.cfg`. If a lower cutoff were used, the saved CSV would only contain brighter stars, and a later run with a higher cutoff would silently get fewer background stars than expected without any error. By always fetching to 20.0, the CSV is safe to reuse regardless of what `magnitude_cutoff` is set to in subsequent runs. The `magnitude_cutoff` is then applied as an in-memory filter on the loaded CSV and can therefore only reduce the set of background stars, never expand it. If background stars fainter than 20.0 are needed, the CSV must be regenerated.

The cone search radius is controlled by `gaia_conesearch_radius_arcsec` in `global.cfg` (default 500 arcsec). The cached CSV contains only stars within the radius that was set when it was fetched. If a CSV is saved to `data/BackgroundStars/` with a small radius and the NIR channel covers a larger field of view, background stars outside that radius will be missing from the simulation. Always ensure the cone search radius is at least as large as the largest channel field of view when pre-fetching background stars.

Gaia queries can be run in asynchronous or synchronous mode via `GAIA_USE_ASYNC_JOBS` in `global.cfg`. In practice, neither mode is reliably fast or stable — if one fails or times out, try switching to the other.

Gaia Query The Gaia query fetches the following parameters: `source_id`, `right_ascension`, `declination`, `parallax`, `gaia_magnitude`, `effective_temperature`, `distance`, `radius`, `mass`, `metallicity`, and `surface_gravity`. These are defined in `GAIA_PROPERTIES` in `src/loaders/load_gaia.py`, analogous to the Excel mapping file. The query joins `gaia_source`, `astrophysical_parameters`, and `astrophysical_parameters_supp` from Gaia DR3, and uses `COALESCE` to retrieve as complete a set of parameters as possible for each source. The priority order in each `COALESCE` expression prefers observed and directly derived quantities over model-based ones, so that background stars with only partial Gaia coverage are still included where possible. These parameter names are used throughout the codebase and mapped directly to the internal parameter dictionary and the `Star` class. Changing them would require updates in many places and is strongly discouraged. To add a new Gaia parameter, add it to `GAIA_PROPERTIES`, add the corresponding alias to the query, and extend the `Star` class and any downstream code that uses it.

Target Star Lookup To query Gaia for the target star, the simulator first resolves the target name via SIMBAD to obtain a Gaia DR3 source ID. If SIMBAD fails or returns no Gaia ID, a small cone search of 6 arcsec around the RA/Dec from the Excel catalogue is used to find the closest source. Once the source ID is known, the full parameter query is run against that specific source ID.

The resolved source ID is also used when fetching background stars — the target itself is identified by source ID and excluded from the background star catalogue. If the source ID cannot be resolved, the target star may end up included as a background star, which has happened in cases where the RA/Dec in the Excel catalogue differed by more than 6 arcsec from the Gaia position. If background star results look suspicious, verifying the source ID resolution in the log file is recommended.

Parameter Enrichment After loading from the Excel catalogue and optionally merging Gaia parameters, the simulator applies a series of fallback derivations to complete the parameter set before initialising the target.

- **Distance** — derived from Gaia parallax as $d = 1000/\varpi$ if missing.
- **Radius** — estimated from Gaia G magnitude, distance, and effective temperature via the Stefan–Boltzmann relation if missing.
- **Spectral type** — inferred from effective temperature using the Mamajek dwarf star table if missing.
- $\log R'_{\mathrm{HK}}$ — assigned from a temperature threshold defined in `global.cfg` (`log_r_teff_threshold`, `log_r_hot_value`, `log_r_cool_value`) if missing.

Effective temperature and mass are not derived. If effective temperature is missing after Excel and Gaia lookup, the simulator stops for the target star, as it is required for all downstream processing. Mass is optional and stays `None` if not provided.

3.6. Background

When `background_type = calc` is set, the sun position used to compute the zodiacal light distribution is taken from the actual wall-clock time of the simulation run (`Time.now()`), not from a configurable observation epoch. Two runs of the same target on different dates will therefore produce different background levels. If reproducibility is required, replace `Time.now()` in `generate_background_calculated_image()` with a fixed Julian date.

3.7. Logging

Each simulation run creates a unique output directory under `output/`, named by timestamp. The log file is placed in that directory and named `cadence_simulator_<timestamp>.log`. It contains the complete record of the run, including configuration loading, parameter enrichment, flux pipeline steps, detector image generation, and background star processing.

3.7.1. Background Stars

During simulation runs it is often useful to know where background stars are located on the detector and what their separation from the target is. A reference Excel file is provided at `<repo_root>/documents/reports/target_back_stars.xlsx`, which contains all background stars for all targets currently in the target Excel catalogue. For each star the Gaia source ID, magnitude, separation, and position are listed. This file was generated by first pre-fetching background stars using `gaiahelper_create_background_stars.py`, which saves one CSV per target, and then combining all CSVs into a single Excel file using `generate_master_excel.py`. The resulting file can be used to diagnose simulated images without having to run the simulator or query Gaia.

Alternatively, all background stars that were loaded and accepted or skipped for a given run are logged during startup. Searching the log for `Background star added` or `Background star skipped` gives the full list including Gaia magnitude, effective temperature, radius, mass, position, offset and Euclidean distance in arcseconds from the target. The offset and separation are particularly useful when diagnosing whether a background star falls within the slit or detector field of view.

```
2026-06-11 13:44:05,975 [INFO] load_background_stars.py:199 Background star added:
star_id_formatted=2 064 323 877 037 081 984 star_id=gaia_2064323877037081984 mag=7.094
teff=4250 radius=142.080 mass=6.611 ra=307.900301 dec=39.880098 dist=739.41 dx=111.613
dy=-211.719 sep=239.338

2026-04-11 22:15:30,118 [INFO] load_background_stars.py:219 Background star skipped
gaia_2033125337719861888. Missing: ['effective_temperature', 'radius',
'log_r', 'spectral_type']
```

3.7.2. Spectroscopy Channels

To find background stars on the detector during a run for the spectroscopy channels, search the log for `Background Stars in Slit`. These entries are logged once per frame and contain the frame index, channel name, roll angle range, total number of background stars in the slit out of the full catalog, and for each star in the slit: the Gaia source ID, G magnitude, effective exposure time in seconds (how long that star was inside the slit during that frame given the roll angle motion), and the detector row range it was spread across:

```
2026-04-11 22:21:07,314 [INFO] background_star_spectroscopy.py:114 Background Stars in Slit,
rendering frame with roll_angle: frame=0 channel=NUV roll_angle_start=0 roll_angle_stop=20
n_in_slit=4/583 star_ids_mag_texp_in_slit=[
ID=gaia_2033123654092583168 | G=14.98277473449707 | EXP=91.579s | YROW=192-194 ;
ID=gaia_2033123477956581632 | G=15.21379566192627 | EXP=218.182s | YROW=69-71 ;
ID=gaia_2033123482293884160 | G=15.598587989807129 | EXP=57.188s | YROW=29-30 ;
ID=gaia_2033123477965613440 | G=15.868956565856934 | EXP=149.219s | YROW=17-18]
```

3.7.3. NIR Photometry Channel

For the NIR photometry channel, search the log for `Background Stars on Detector`. Since the NIR field of view is much larger than the spectroscopy slit, many more stars can appear simultaneously. The log entry contains the frame index, channel name, roll angle range, number of stars on the detector out of the full catalog, and for each star: the Gaia source ID, G magnitude, and PSF footprint in pixel column and row ranges. The column and row ranges reflect the extent of the PSF stamp on the detector, not just a center position.

The first frame logs all stars currently on the detector:

```
2026-04-11 22:27:18,438 [INFO] background_star_photometry.py:95 Background Stars on
Detector, rendering frame with roll_angle: frame=0 channel=NIR roll_angle_start=0
roll_angle_stop=4 n_on_detector=29/49 stars=[
ID=gaia_2064323877037081984 | G=7.094086647033691 | XCOL=406-419 | YROW=61-68 ;
ID=gaia_2064328343803107072 | G=9.303207397460938 | XCOL=168-178 | YROW=397-407 ;
ID=gaia_2064327690968950656 | G=11.62588119506836 | XCOL=58-62 | YROW=202-220 ;
ID=gaia_2064328584321277440 | G=11.786202430725098 | XCOL=202-218 | YROW=491-498 ;
ID=gaia_2064323567799449984 | G=11.817398071289062 | XCOL=191-209 | YROW=0-9 ;
ID=gaia_2064329851334709888 | G=12.094717025756836 | XCOL=422-428 | YROW=162-169 ;
ID=gaia_2064328549961538944 | G=12.844476699829102 | XCOL=154-165 | YROW=413-425 ;
ID=gaia_2064328343803103488 | G=12.915674209594727 | XCOL=199-208 | YROW=386-394 ;
ID=gaia_2064329993070516480 | G=13.180278778076172 | XCOL=505-506 | YROW=268-281 ;
ID=gaia_2064333566483329920 | G=13.53044605255127 | XCOL=342-358 | YROW=481-483 ;
....]
```

Subsequent frames log only changes: stars that entered or left the detector to keep the log manageable:

```
2026-04-11 22:22:51,802 [INFO] background_star_photometry.py:116 Background Stars on
Detector, rendering frame with roll_angle: frame=2 channel=NIR roll_angle_start=10.6667
roll_angle_stop=14.6667 n_on_detector=399/583
added=[ID=gaia_2033121386349879808 | G=14.405774116516113 | XCOL=41-48 | YROW=0-8 ;
ID=gaia_2033148015149030912 | G=12.988380432128906 | XCOL=557-568 | YROW=501-512 ;
ID=gaia_2033121420709606400 | G=13.088274955749512 | XCOL=63-69 | YROW=0-6 ; ...]
removed=[ID=gaia_2033110734829849856 ; ID=gaia_2033122348422445312 ;
ID=gaia_2033127983419753600 ; ID=gaia_2033147774630804608 ; ID=gaia_2033110730500140672 ;
ID=gaia_2033127261865235328 ; ID=gaia_2033110700470081920 ; ID=gaia_2033110872268757376 ;
...]
```

4. Extending the Simulator / Performance Considerations

4.1. Adding Star and Planet Parameters

To add a new stellar parameter to the simulation, the following steps are required:

1. Add the parameter as a column to the Excel target catalogue.
2. Add the mapping from the Excel column name to the internal parameter name in the `[stars]` section of `excel_mapping.cfg`. If the parameter is required for the simulation to run, add it to `[required_stellar_parameters]` as well.
3. Add the parameter as a field to the `Star` class in `src/domain/star.py`.
4. If the parameter can be fetched from Gaia, add it to `GAIA_PROPERTIES` in `src/loaders/load_gaia.py` and add the corresponding alias to the query in `_gaia_select_joined_base()`.
5. If a fallback derivation is possible when the parameter is missing, implement it in `src/loaders/load_stellar_and_planetary_properties.py`.

For planetary parameters, the process is the same but using the `[planets]` section of `excel_mapping.cfg` and the `Planet` class. Note that the planet is currently only used to store properties loaded from the Excel catalogue and is not yet used in any simulation calculations.

4.2. Extending Channel and Global or User Configuration

Adding a channel parameter Channel parameters are defined in the `Channel` base class in `src/configs/channel_config.py`. `SpectroscopyChannel` and `PhotometryChannel` both inherit from it. Decide first whether the new parameter applies to all channels (`Channel`), spectroscopy only (`SpectroscopyChannel`), or photometry only (`PhotometryChannel`), and add the field to the appropriate class.

Note that `Channel` is never instantiated directly — only `SpectroscopyChannel` and `PhotometryChannel` are. This means even if the parameter is defined on `Channel`, it must be passed when constructing a `SpectroscopyChannel` or `PhotometryChannel` instance in `src/loaders/load_channel.py`, where the channel config files are parsed and the channel objects are created.

Adding a global config parameter Global configuration parameters are defined in the `GlobalConfig` dataclass in `src/configs/global_config.py`. To add a new parameter, add it to the dataclass, parse it from the config file in `_read_global_cfg()`, and pass it to the `GlobalConfig` constructor. `load_global_config()` must be called exactly once at startup — it reads the config file and caches the result. Everywhere else in the codebase, use `get_global_config()` to access it. Never call `load_global_config()` more than once, as this would create a second instance and the cached singleton would not be updated.

User config and run context The user config (`UserConfig`) is loaded once at startup and passed through explicitly rather than accessed globally. It contains the target name and exposure times per channel. The target name is stored in the `RunContext` after initialisation and passed through the pipeline from there. Exposure times are read from the user config and written directly into the channel objects during channel initialisation, so they do not need to be accessed separately afterwards. In practice, the user config is consumed during startup and the `RunContext` carries the relevant information for the rest of the run.

`RunContext` is a small frozen dataclass containing the target name, the output directory for the current run, and the run timestamp. It is created once at startup and passed to all pipeline stages that need to write output or log target-specific information.

4.3. Adding Additional ISM Absorption Lines

Additional ISM absorption lines can be added by extending the existing ISM line and absorber definitions.

For implementation details and required updates to the wavelength-window optimisation, see Section 5.1.2.

5. Caches and Performance Improvements

5.1. ISM Absorption Optimisation / Voigt Profile Cache

The ISM absorption calculation was identified as a performance bottleneck because the Voigt profiles were evaluated over the full wavelength grid. The implemented ISM model only affects the wavelength region around the Mg II, Mg I and Fe II line centres. The calculation was therefore restricted to this region + margin of 200A.

```
line_centers = [
    lineMg1['wave'],
    lineMg2['wave'],
    lineMgI['wave'],
    lineFeII['wave']
]

wl_min = min(line_centers) - SPECTRAL_WINDOW_MARGIN_A
wl_max = max(line_centers) + SPECTRAL_WINDOW_MARGIN_A

wl = flux[:, 0]
mask = (wl >= wl_min) & (wl <= wl_max)

if not np.any(mask):
    return flux

wl_ism = wl[mask]
```

The thought process behind this change was that the implemented ISM absorption only contains these specific transitions. Outside the selected wavelength window, the transmission of the implemented model is treated as one. Leaving the flux unchanged outside the mask therefore gives the same result as multiplying it by a transmission of one.

Only the affected part of the flux array is modified:

```
flux_absorption = n_flux.copy()
flux_absorption[mask] = ISM * n_flux[mask]
```

This keeps the output identical to the full-grid calculation for the implemented ISM model, while reducing the number of wavelength samples passed through the Voigt profile calculation.

If additional ISM species are added later, their line centres must also be included in the window definition. Otherwise the mask could exclude wavelengths where newly implemented absorption should be applied.

5.1.1. Voigt Profile Cache

The Voigt profile calculation was also cached. The cache key contains only the quantities that determine the shape of the Voigt profile:

```
cache_key = (
    wavelength.tobytes(),
    float(absorber["B"]),
    float(absorber["Z"]),
    float(line["wave"]),
    float(line["gamma"]),
)
```

The cache stores the Voigt profile shape, not the final optical depth. The profile shape is determined by the wavelength grid (wavelength), the Doppler broadening parameter (B), the absorber velocity shift (Z), the line centre (wave) and the damping parameter (gamma). The absorber column density (N) is applied only after the cached profile has been calculated, when the optical depth is computed. It is therefore not part of the cache key. Since N is not used during the Voigt profile calculation, it must not be included in the cache key.

The cache implementation is independent of the number of supported ISM transitions. Newly added lines automatically generate separate cache entries through their line centre and damping parameters. **No cache modifications are required when adding additional ISM lines.**

The cache implementation only needs to be revisited if additional parameters are introduced that modify the Voigt profile shape itself. Simply adding new ISM transitions does not require changes to the cache implementation.

5.1.2. Adding Another ISM Absorption Line

Additional ISM absorption lines can be added to the existing line core absorption model, but all parts of the calculation must be updated consistently.

First, define the additional line parameters. The line centre must be added to the list used for the ISM wavelength window:

```
line_centers = [
    lineMg1['wave'],
    lineMg2['wave'],
    lineMgI['wave'],
    lineFeII['wave'],
    lineNew['wave']
]
```

The corresponding absorber and line definition must then be passed to the Voigt profile calculation:

```
ISMNew = voigtq(wl_ism, absorberNew, lineNew)
```

The new transmission must also be included in the total ISM transmission:

```
ISM = ISMMg21 * ISMMg22 * ISMMg1 * ISMFe2 * ISMNew
```

The important point is that every newly implemented ISM line must be included in the wavelength window. If the line centre is not added to `line_centers`, the mask may exclude the relevant wavelength region and the new absorption line will not be applied.

The Voigt profile cache does **not** need to be changed when adding a normal line that uses the existing profile calculation.

5.2. Global Configuration Cache

The global configuration must be loaded exactly once during program startup and is subsequently reused through a module-level cache.

```
def load_global_config(path: Path) -> GlobalConfig:
    global _GLOBAL
    if _GLOBAL is None:
        _GLOBAL = _read_global_cfg(path)
    return _GLOBAL
```

Configuration files must not be reloaded during normal program execution. Components requiring configuration values must access the already loaded configuration object through:

```
cfg = get_global_config()
```

All configuration access throughout the codebase must use this mechanism. This guarantees that every component operates on the same configuration instance and avoids repeated configuration file parsing and validation.

5.3. User Configuration Cache

see above Section 5.2. Same style, but usually i passed the user config around, since it was not often used. However, it is later more often used, you can use it similarly to `globalConfig()`.

5.4. Stellar Model Caches

Stellar model loading is centralised in `load_model_for_temperature()`. All code requiring stellar model data must use this function and must not load model files directly.

The model loading system uses three caches:

```
_MODEL_CACHE = {}
_AVAILABLE_MODELS = None
_CUT_MODEL_CACHE = {}
```

Available Model Cache The available model directories are scanned only once. During the first call, the contents of `data/models` are inspected and the result is stored in `_AVAILABLE_MODELS`. Subsequent calls reuse the cached directory list and do not perform another filesystem scan.

Model Cache The raw stellar model data loaded from disk is stored in: `_MODEL_CACHE`. The cache key is the full model file path. As a result, the cache is independent of the model selection logic. Changes to the available stellar models or temperature matching algorithm do not require modifications to the cache implementation, provided that each model continues to be identified by a unique file path.

Cut Model Cache The wavelength-trimmed model data is stored in: `_CUT_MODEL_CACHE` After loading, the model spectrum is restricted to the wavelength range obtained via the cfgs of nuv, vis and nir required by the simulation:

```
cut_model_data = _cut_model_wavelength_range(full_model_data, wl_min_A, wl_max_A)
```

The resulting array contains the wavelength grid and the corresponding model values within the selected wavelength range. This processed array is stored in the cache and reused by later requests.

The cache therefore avoids repeated wavelength trimming and repeated model file loading.

The cached arrays are marked as read-only:

```
cut_model_data.setflags(write=False)
```

Code receiving model data from `load_model_for_temperature()` must treat the returned array as immutable. Any modification requires creating a copy first, which we already do at the beginning of the flux pipeline.

This cache is especially important for simulations with many background stars, where the same stellar model can be requested repeatedly.

5.5. Unred Curve Cache

The extinction curve construction in `cute_unred` is cached through: `_UNRED_CURVE_CACHE`. The cache stores the wavelength-dependent extinction curve. The cache key consists of:

```
cache_key = (wave.tobytes(), float(R_V), bool(LMC2),bool(AVGLMC))
```

The cache therefore depends only on the wavelength grid and the selected extinction-law parameters. The extinction curve is independent of both the input flux and the colour excess E(B-V).

The expensive spline-based extinction curve construction is performed only once for a given wavelength grid and extinction-law configuration. Subsequent calls reuse the cached extinction curve and only apply the final E(B-V) scaling.

The cache key includes all quantities used during extinction-curve construction. Changes to stellar models, wavelength sampling, or extinction-law configuration automatically generate separate cache entries and therefore do not require modifications to the cache implementation.

5.6. Additional Caches

5.6.1. Mamajek Table Cache

The Mamajek spectral type table is loaded once and cached in `_MAMAJEK_CACHE` in `src/loaders/load_stellar_and_planetary_properties.py`. Subsequent calls to `infer_mamajek()` reuse the cached table without re-reading the file.

5.6.2. Excel Mapping Cache

The Excel mapping configuration is loaded once and cached in `_EXCEL_MAPPING_CACHE` in `src/loaders/load_stellar_and_planetary_properties.py`. Subsequent calls to `load_excel_mapping()` reuse the cached mapping without re-parsing `excel_mapping.cfg`. This is particularly relevant for simulations with large background star catalogues, where `load_excel_mapping()` is called once per background star.

5.6.3. Background Star Visibility Log Cache

`_LAST_VISIBLE_SIGNATURE_BY_CHANNEL` in `src/instrument/background_star_photometry.py` is not a performance cache. It tracks which background stars were visible on the detector in the previous frame per channel, so that the log only reports changes rather than repeating the full list every frame.

5.6.4. Profile Spread Warning Cache

`_PROFILE_SPREAD_WARNED_CHANNELS` in `src/instrument/spectrum_spread.py` is not a performance cache. It ensures that the cross-dispersion profile spread conservation warning is logged at most once per channel per run.

6. Troubleshooting

This section collects common problems and their likely causes, with references to the detailed explanation elsewhere in the manual. If a fix is not obvious from the cause given here, check the log file (see Section 3.7) for the detailed error message first.

6.1. Target and Excel Catalogue

Problem	Likely Cause
Target not found, despite the name appearing correct	Hidden characters in the `pl_name` cell, often introduced when copying from external sources, or the planetary suffix included in `target_name` (only `KELT-9`, not `KELT-9 b`, is accepted) (Section 3.3)
Simulator stops immediately at startup with an Excel-related error	More than one `.xlsx` file present in `input/` (Section 3.3)
Missing required stellar parameter, simulator stops before simulation starts	Parameter not present in the Excel catalogue, not retrievable from Gaia, and no fallback derivation applies (Section 1.2.3)

6.2. Gaia

Problem	Likely Cause
Gaia query is slow, times out, or fails intermittently	Try switching `GAIA_USE_ASYNC_JOBS` in `global.cfg` to the other mode (Section 3.5)
Background stars missing from the simulation that should be visible	Cached CSV in `data/BackgroundStars/` was fetched with a cone search radius smaller than the channel's field of view (Section 3.5)
Target star also appears as a background star	Gaia source ID resolution mismatch, typically when the Excel RA/Dec differs from the Gaia position by more than 6 arcsec (Section 3.5)
Background star silently missing from the catalogue without an error	Required parameters not available from Gaia for that star; missing background stars are skipped rather than stopping the run.

6.3. Configuration

Problem	Likely Cause
Simulator stops with a spread/PSF configuration error	Neither `spread_profile_file` nor `spread_half_height_pix` configured for a spectroscopy channel (Section 2.2.2)
Background level differs between two runs of the same target	`background_type = calc` uses the wall-clock time of the run, not a fixed observation epoch (Section 3.6)
Simulator stops at startup with a missing or invalid value (boolean, integer, or float) error	A configuration value in `global.cfg` or a channel `.cfg` file is missing, mistyped, or has an unexpected format
Simulator stops because no channel is enabled	`run_vis`, `run_nuv`, and `run_nir` are all set to `False` in `global.cfg`; at least one must be enabled
Simulator stops with a range or ordering error (e.g. minimum greater than maximum)	A pair of related `global.cfg` values is inconsistent, for example `cosmic_rays_min` > `cosmic_rays_max`, or `log_r_hot_value` > `log_r_cool_value`

In general, any error encountered while parsing or validating a configuration file is reported to the console with

a short message indicating what is wrong; the log file contains the full detail (see Section 3.7). The log file is, in general, very verbose and contains substantially more information than what is printed to the console, so it is the first place to check for any issue beyond what the console message already explains.

6.4. Background Stars

Problem	Likely Cause / How To Check
Need to confirm whether a specific background star was included or skipped	Search the log for `Background star added` or `Background star skipped`, filtering by Gaia source ID (Section 3.7.1)
Need to find where a specific background star landed on the detector for a spectroscopy channel	Search the log for `Background Stars in Slit` and look for the star's Gaia source ID in the per-frame listing (Section 3.7.1)
Need to find where a specific background star landed on the detector for the NIR channel	Search the log for `Background Stars on Detector`; only the first frame logs the full list, subsequent frames log only additions and removals (Section 3.7.1)

6.5. Batch Runs

Problem	Likely Cause
`starrunner.sh` exits immediately with no output and no error	The given directory's `.txt` files are direct children, not inside subdirectories; pass that directory itself as the leaf target (Section 3.1.3)

7. Tests

The test suite consists of unit tests and snapshot tests. Snapshot tests compare freshly computed simulator output against previously saved reference arrays produced by a pipeline run. They guard against unintended numerical changes when the pipeline is modified. If a change intentionally alters numerical output, the snapshots must be regenerated.

Snapshot tests are the most important test category, as they freeze the pipeline output end-to-end. Effective area files and all configuration files are copied into the snapshot directory (as npz) alongside the reference arrays, so changes to those files do not break the tests. Only implementation changes require regenerating snapshots — which is correct behaviour, since a changed implementation may either fix a bug or introduce one, and the snapshots should reflect the intended output.

7.1. `test_flux_pipeline.py`

This suite covers the spectral modelling steps from stellar atmosphere model through to photon flux density at Earth. Each pipeline stage is tested independently: stellar intensity to luminosity conversion, chromospheric line core emission, interstellar medium absorption, geometric dilution to flux at Earth, dereddening, and conversion to photon flux density. Tests are run for WASP-69, WASP-189, and HD 2685. Snapshots are plain text files written by the simulator when `write_intermediate_arrays = 1` is set in `configs/global.cfg`.

7.2. `test_detector_pipeline.py`

This suite covers the instrument simulation steps from photon flux density through convolution with the instrument response, spreading to 2D detector images, and science frame assembly. All three channels (NUV, VIS, NIR) are tested for KELT-9 and HD 2685. Snapshots are `.npz` files written by the simulator when `write_intermediate_arrays = 1`.

7.3. Regenerating Snapshots

1. In `configs/global.cfg`, set `write_intermediate_arrays = 1` and `orbit_revolutions = 0.1` (only the intermediate array files are needed; a full orbit is not required).
2. Run the simulator for the targets relevant to the changed pipeline stage.

   For `test_flux_pipeline.py`: WASP-69, WASP-189, and HD 2685.

   For `test_detector_pipeline.py`: KELT-9 and HD 2685.
3. Copy all `.txt` snapshot files (used by `test_flux_pipeline.py`) and all `.npz` snapshot files (used by `test_detector_pipeline.py`) from each output directory into `tests/snapshot_tests/snapshots/`.

   The `.npz` binary format is used for detector snapshots because detector images are large and loading them as text would make the tests prohibitively slow.
4. Run the simulator for the targets relevant to the changed pipeline stage. For `test_flux_pipeline.py`: WASP-69, WASP-189, and HD 2685. For `test_detector_pipeline.py`: KELT-9 and HD 2685.
5. Verify the full test suite passes: `python -m pytest tests/`
6. Reset `write_intermediate_arrays = 0` and `orbit_revolutions` to the original value for normal simulation runs.